

TDC-GPX FSM Atlas

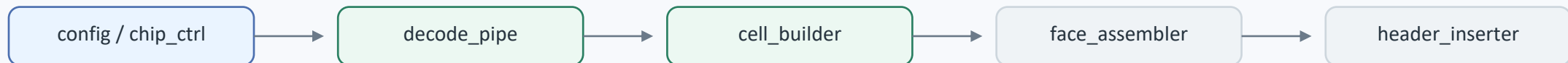
업데이트된 RTL/VHDL 코드만 기준으로 정리한 FSM 상태도와
모듈 간 상관관계, 그리고 추가 보완 검토

Control / bus

Data / image

Recovery / error

명시적 enum FSM이 있는 모듈을 중심으로 정리했고,
state가 없는 모듈은 별도 슬라이드에서 implicit controller/stream
pipeline으로 구분했다.



[FSM diagrams](#) • [interaction map](#) • [review](#)

FSM 인벤토리와 연결 구조

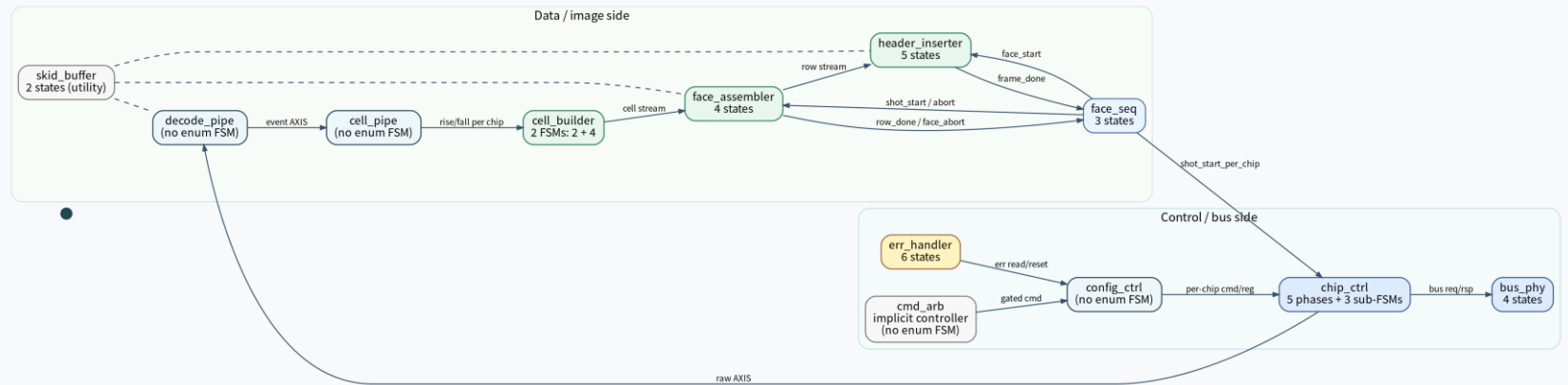
명시적 FSM이 있는 모듈과, state 없이 동작하는 pipeline/controller를 분리해서 본 전체 지도

명시적 FSM이 있는 모듈

- bus_phy, chip_ctrl, chip_init, chip_reg, chip_run, err_handler
- cell_builder(collect/output), face_seq, face_assembler, header_inserter, skid_buffer

FSM이 없는 모듈

- config_ctrl, decode_pipe, raw_event_builder, cell_pipe, output_stage
- cmd_arb는 enum state 대신 pending bit + timeout으로 동작



enum FSM

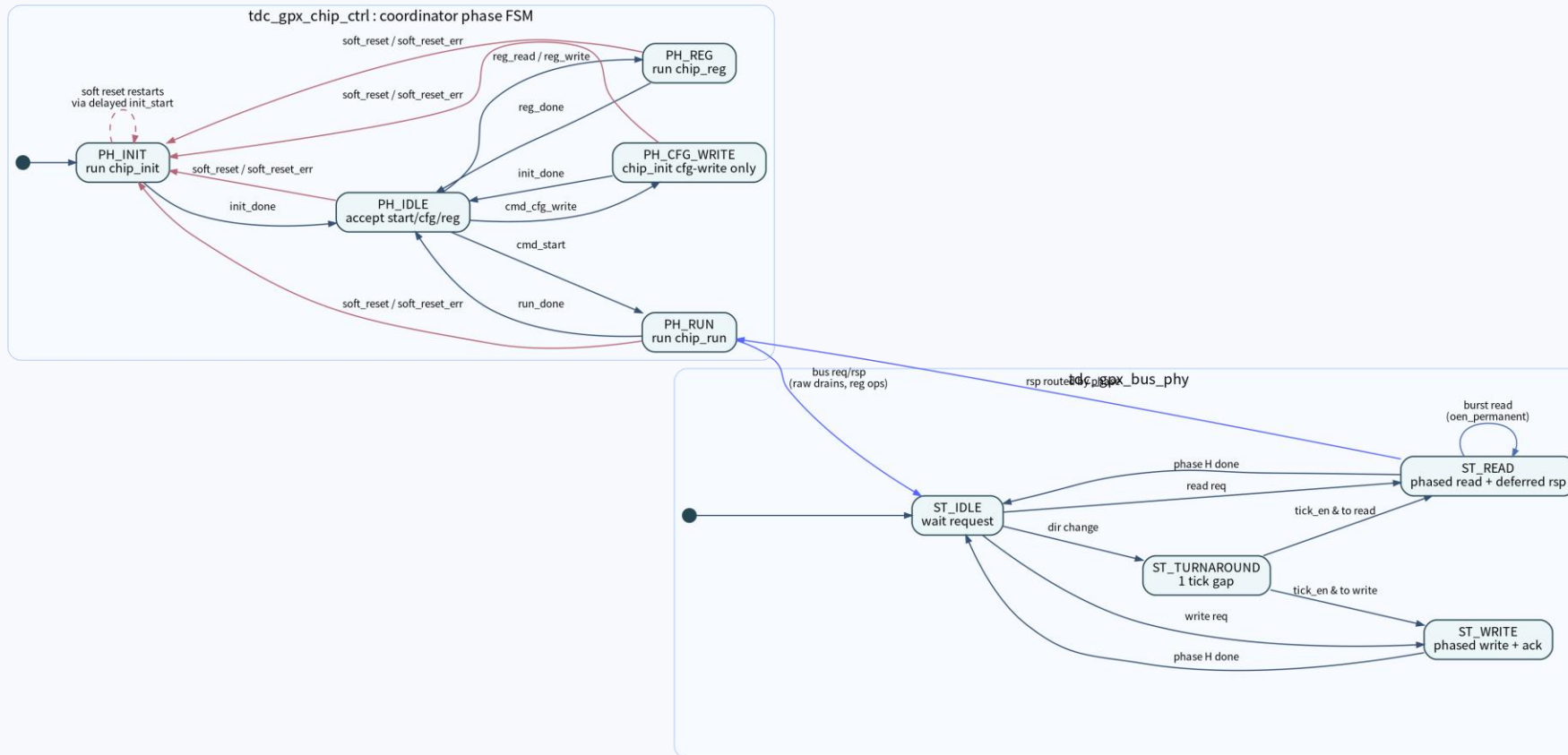
stream / utility

error / recovery

이 슬라이드는 “어디에 state가 있고, 어떤 event가 어느 모듈을 건드리는가”를 한 번에 보는 용도다.

Control front-end: chip_ctrl + bus_phy

chip_ctrl는 sub-FSM dispatch를 담당하고, bus_phy는 실제 TDC bus phase를 생성한다.



읽는 포인트

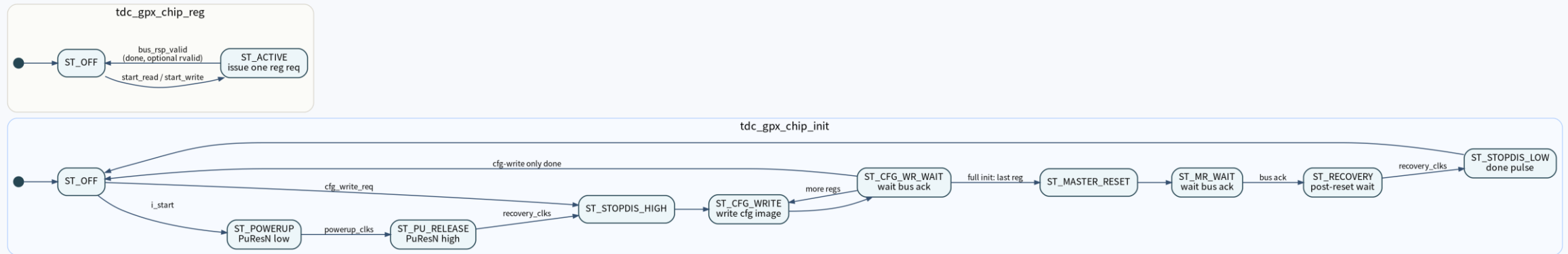
- PH_IDLE에서 start / cfg_write / reg access 우선순위를 결정한다.
- bus_phy는 IDLE / READ / WRITE / TURNAROUND 4-state로 phase timing을 만든다.

현재 검토 포인트

- soft_reset 시 이전 response를 비우는 전용 phase가 없어, stale response quarantine를 FSM 레벨에서 더 명확히 하는 편이 안전하다.

Small control FSMs: chip_init / chip_reg

초기화 시퀀스와 개별 register access는 분리된 sub-FSM으로 유지된다.

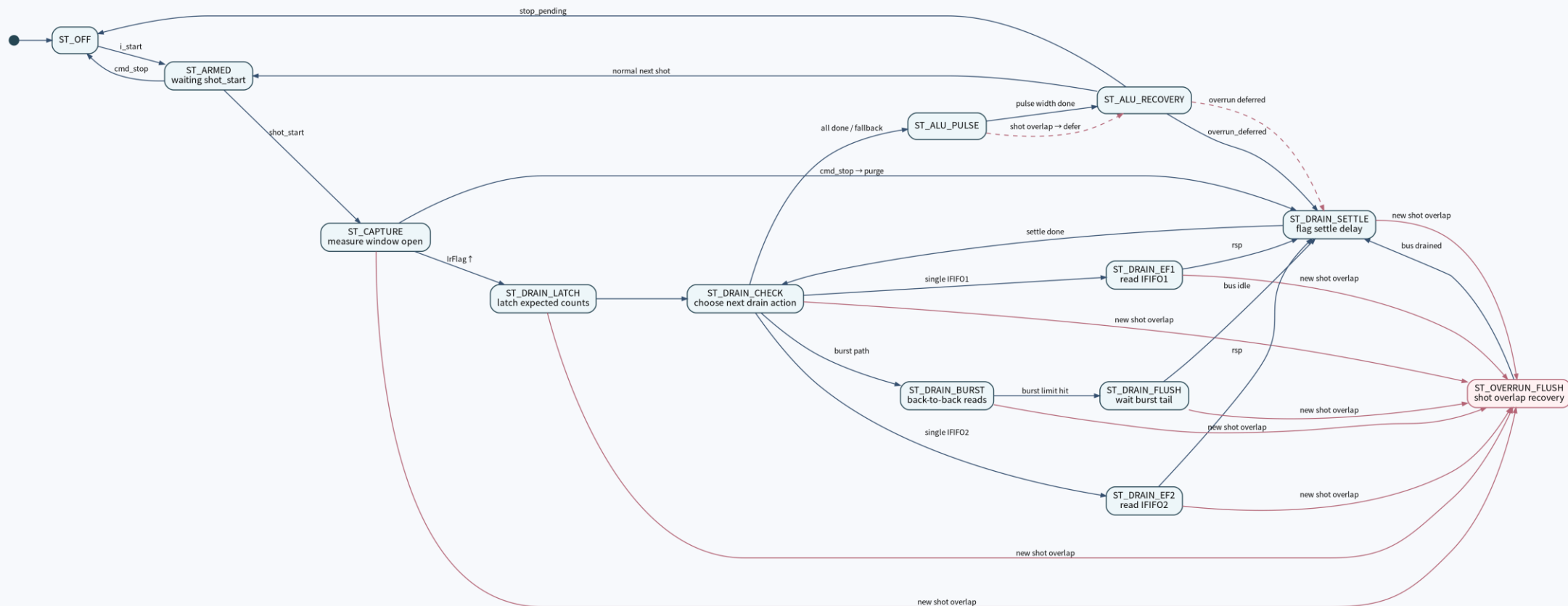


핵심 포인트

- chip_init은 full init(powerup+master reset)과 cfg-write-only 경로를 같은 FSM에서 분기한다.
- chip_reg는 ST_OFF ↔ ST_ACTIVE 2-state로 1 request / 1 response를 표현한다.
- 이 둘의 ordering 보장은 chip_ctrl phase와 bus routing에 의존한다.

chip_run: capture → drain → ALU → overrun recovery

가장 상태 수가 많은 per-chip run FSM이며, capture window와 IFIFO drain, ALU pulse, overlap 복구를 한 곳에 가진다.



핵심 구조

- IrFlag↑ 이후 drain check가 burst / single / done 경로를 선택한다.
- overrun은 ST_OVERRUN_FLUSH 또는 ALU 이후 deferred path로 흘러간다.

추가 보완 검토

- 현재 raw backpressure는 chip_ctrl의 1-deep hold까지만 확실하고, chip_run은 ST_DRAIN_CHECK 밖에서는 producer 관점 ready/valid 계약이 완전하지 않다.

err_handler: automatic recovery FSM

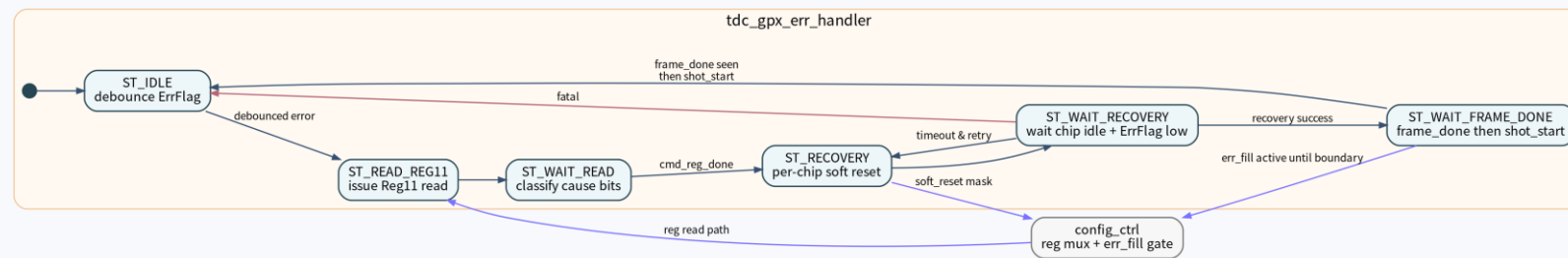
debounce → Reg11 read → per-chip recovery → frame/shot boundary resync 흐름을 담당한다.

구조

- error chip mask를 잡고 Reg11을 읽어 원인을 분류한 뒤 soft reset을 발행한다.
- 정상 회복 후에는 frame_done + 다음 shot_start 경계까지 err_fill을 유지한다.

추가 보완 검토

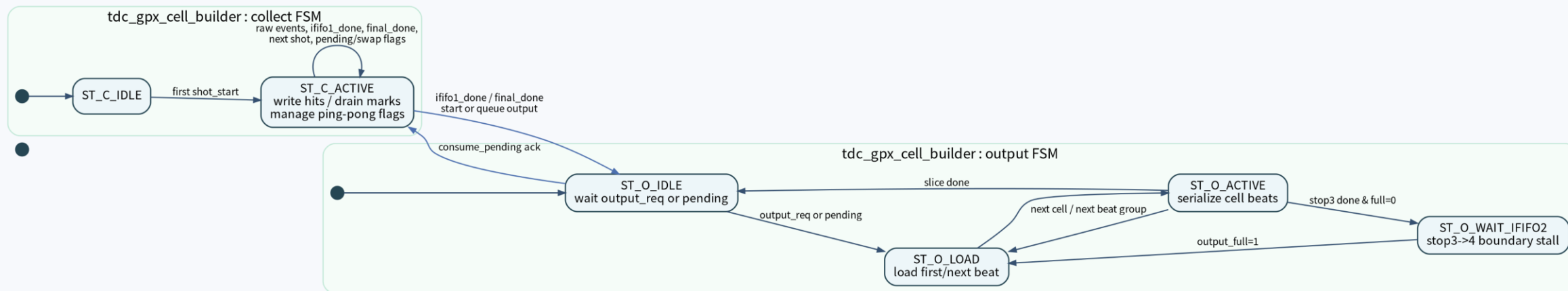
- fatal로 ST_IDLE에 떨어질 때 err_fill / chip_mask / cause를 어떻게 정리할지 명시적 cleanup 경로가 있으면 더 안전하다.



config_ctrl는 FSM은 없지만, err_handler의 reg mux와 err_fill gating을 실제 버스/데이터 경로에 반영하는 핵심 중재점이다.

cell_builder: collect/output dual FSM

collect와 output을 분리했지만, shot overlap과 buffer ownership은 여전히 이 모듈의 핵심 난점이다.



읽는 포인트

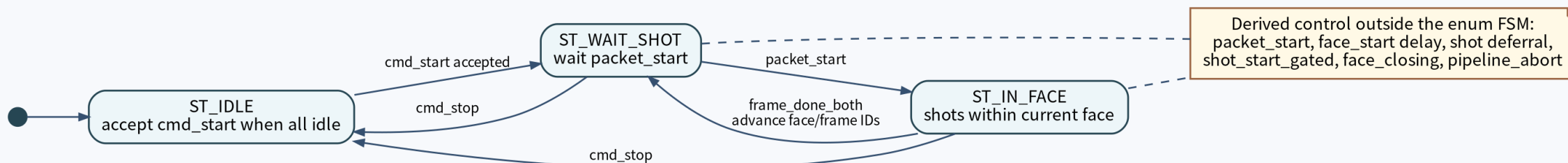
- collect FSM은 사실상 ST_C_IDLE → ST_C_ACTIVE 한 번만 거친 뒤, 이후 shot 경계를 state가 아니라 flag로 처리한다.
- output FSM은 stop 3→4 경계에서 ST_O_WAIT_IFIFO2로 stall할 수 있다.

추가 보완 검토

- output busy 상태에서 새 shot이 들어오면 deferred swap bit로 버티기 때문에, buffer owner를 별도 FSM(또는 3-buffer)로 분리하는 편이 freeze 관점에서 훨씬 명확하다.

face_seq: face / frame / shot sequencer

핵심 FSM은 3-state지만, 실제 동작 의미의 상당 부분은 packet_start·shot deferral·closing logic에 있다.

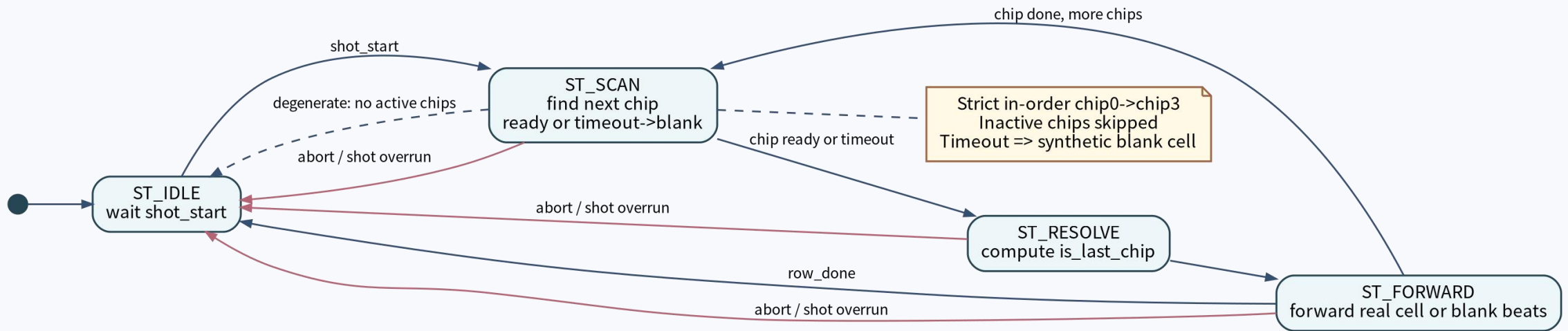


핵심 포인트

- ST_IDLE에서 start acceptance 조건이 가장 엄격하다: chip busy, assembler/header idle, 각종 tvalid까지 모두 확인한다.
- shot_start_gated / pipeline_abort / face_closing은 enum state 밖의 derived logic이므로, 실제 reasoning은 이 로직까지 함께 봐야 한다.

face_assembler: strict in-order row assembly

chip0→chip3 순서를 강제하고, 준비되지 않은 chip은 timeout 후 blank cell로 채운다.

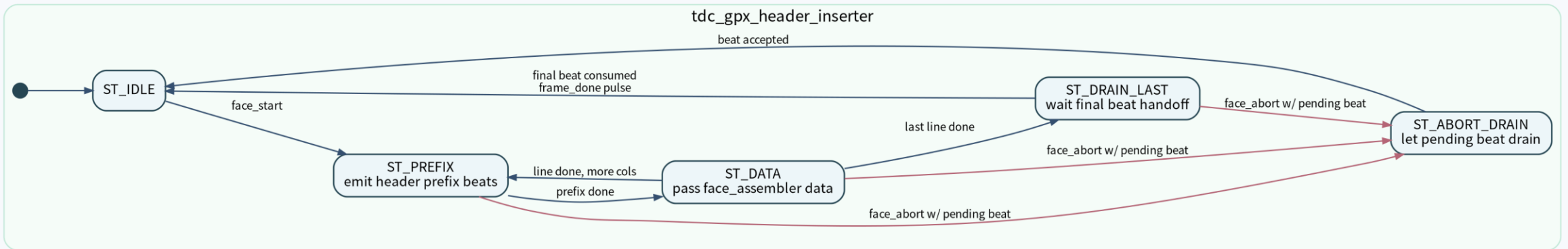
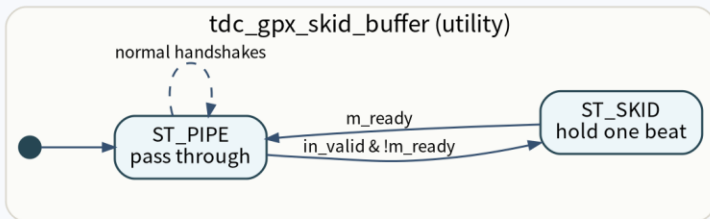


핵심 구조

- ST_SCAN에서 ready chip 또는 timeout blank를 고르고, ST_RESOLVE에서 마지막 chip 여부를 확인한 뒤 ST_FORWARD로 들어간다.
- shot overrun이나 external abort는 현재 face를 즉시 잘라 ST_IDLE로 복귀한다.

header_inserter + skid_buffer

출력 마지막 beat handoff와 abort-drain을 명시적으로 다루는 FSM, 그리고 전 구간에서 재사용되는 2-state utility buffer



핵심 포인트

- header_inserter는 ST_DRAIN_LAST / ST_ABORT_DRAIN을 통해 마지막 beat와 abort 시점을 명시적으로 분리한다.
- skid_buffer는 ST_PIPE / ST_SKID 2-state로 backpressure를 흡수하는 공통 유틸리티다.

명시적 FSM이 없는 모듈들

이 모듈들은 stream transform, counters, pending bits, or wiring 중심으로 동작하므로 상태도보다 contract/flag를 보는 편이 맞다.

config_ctrl

chip_ctrl/err_handler/cmd_arb를 묶는
mux·fanout·status integration

cmd_arb

enum state 대신 reg_active / pending /
timeout counter로 제어

decode_pipe

decoder_i_mode + raw_event_builder +
skid 조합

raw_event_builder

raw word → per-stop event 생성, abort 입력 있음

cell_pipe

rise/fall demux + cell_builder array wrapper

output_stage

rise/fall output cluster container
(face_assembler + header_inserter)

status_agg

sticky status / error count / timestamp
aggregation

csr_pipeline / top

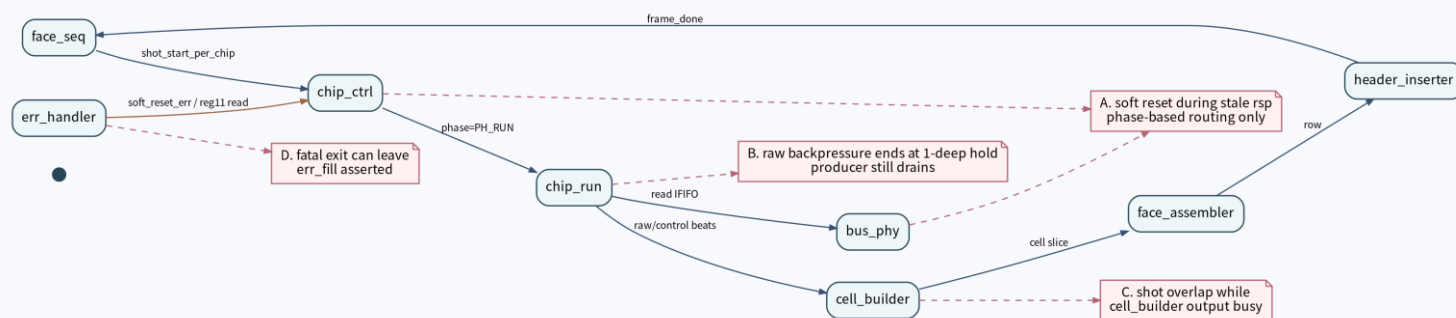
CSR sanitize, snapshot, top-level routing /
glue logic

해석 방법

- 이 그룹은 “상태도”보다 인터페이스 contract와 flush / ready / pending bit를 먼저 보는 편이 낫다.
- 특히 cmd_arb와 config_ctrl는 명시적 FSM이 없지만, 전체 동작 정합성에는 매우 큰 영향을 준다.

FSM-to-FSM contract hotspots

지금 남은 핵심은 개별 FSM 숫자보다, 서로 맞물리는 경계 contract를 얼마나 명시적으로 담았는가에 있다.



1 soft reset ↔ stale response
chip_ctrl가 soft reset 후 곧바로 PH_INIT로 복귀하므로, 이전 응답을 버리는 전용 state 또는 epoch tag가 있으면 더 안전하다.

2 raw backpressure source end
chip_ctrl hold register 뒤로는 tready가 보이지만, chip_run producer까지 end-to-end로 잠그는 계약은 아직 약하다.

3 cell_builder ownership / queue
deferred swap + single pending slot 구조는 shot overlap reasoning을 어렵게 만든다.

4 fatal cleanup
err_handler fatal path에서 err_fill / cause / chip mask 정리 정책을 더 명시적으로 둘 필요가 있다.

추가 보완 검토 결론

결론적으로, “로컬 Fix 몇 개”보다 몇 군데의 FSM contract를 한 단계 더 명시화한 뒤 freeze하는 쪽이 안전하다.

필수 before freeze	강한 권장	Freeze 판단
- cell_builder를 owner-driven buffer FSM으로 재구성하거나, 최소한 pending / swap을 명시적 buffer state로 분리 - soft reset 후 stale response를 버리는 RESP_DRAIN state 또는 epoch/tag 체계 추가 - raw path를 producer까지 포함한 end-to-end handshake/FIFO로 닫기 - err_handler fatal 종료 시 err_fill / chip_mask / cause cleanup 정책 확정	- pipeline_abort 뒤에 quiesce_ack를 두어 다음 face/shot 시작을 더 명확히 gating - response 식별 검증을 reg phase 밖(init/run)까지 확대 - cell_builder pending queue 깊이 > 1 또는 overflow policy를 명시	- 현재 수준은 “상태도 정리 완료” 단계이지, 바로 freeze 권고 단계는 아님 - 필수 4개가 닫히면 상당수 위험이 FSM contract 차원에서 정리됨 - 그 이후에는 local bug fix + regression으로 freeze 검토 가능

Bottom line: FSM 개수는 충분하지만, 경계 contract를 더 explicit하게 만들면 수정/검증/회귀가 훨씬 쉬워진다.